# Universidad Politécnica de Pachuca




**“PRACTICA 4”**

**Tecnologías de la Información e Innovación Digital**

**04_01**

**Nombre de la Alumna: Ordaz Cano Mariana**

**Docente: Montoya Cerro Miguel Ángel**

**Pachuca de Soto, Hgo.  a 19 de Octubre del 2026**

# 2. Índices

## Índice de páginas

## Contenido


2. Índices ..... 2
3. Introducción ..... 3
4. Marco Teórico ..... 4
5. Desarrollo ..... 5
6. Conclusiones ..... 13
7. Anexos ..... 13
8. Referencias ..... 13


## Índice de ilustraciones


Ilustración 1: Importación de librerías y paquetes en MainActivity.kt ..... 6
Ilustración 2: Ventana de inicio de Asset Studio para la selección de Vector Assets ..... 6
Ilustración 3: Selección y personalización del icono de corazón (baseline_favorite_24) ..... 7
Ilustración 4: Declaración del estado del contador utilizando remember y mutableStateOf ..... 8
Ilustración 5: : Definición básica de la fila (Row) con el icono de Me gusta ..... 9
Ilustración 6: Adición del evento clickable en el icono para el incremento de contador ..... 9
Ilustración 7: : Inclusión del texto del contador con modificador de padding lateral ..... 10
Ilustración 8: Visualización previa interactiva del contador incrementado en Android Studio ..... 11
Ilustración 9:Código estructurado completo de la función app() ..... 11


## Índice de tablas


Tabla 1: Comparativa entre conceptos de estado y recursos graficos ..... 12

# 3. Introducción

## 1.1 Contextualización

En el desarrollo moderno de interfaces de usuario para Android con Jetpack Compose, la interactividad es un pilar fundamental. A diferencia del enfoque tradicional de Vistas XML donde la interfaz se actualiza de manera imperativa mediante referencias directas a elementos visuales, Jetpack Compose utiliza un paradigma declarativo reactivo basado en el concepto de Estado (State). Cuando el estado de un componente cambia, la interfaz se re-dibuja automáticamente mediante el proceso conocido como recomposición.

## 1.2 Planteamiento del problema u objeto de estudio

Para construir componentes dinámicos como botones de reacción ("Me gusta"), contadores de interacciones o barras de progreso, los desarrolladores deben comprender cómo gestionar variables cuyo valor debe preservarse a través de los ciclos de recomposición. En esta práctica se analiza la integración de Vector Assets mediante la herramienta Asset Studio y la implementación de estados reactivos utilizando las funciones remember y mutableStateOf.

## 1.3 Objetivo general

Desarrollar una interfaz interactiva en Android Studio utilizando Jetpack Compose, incorporando Vector Assets creados desde Asset Studio y gestionando eventos de usuario mediante el manejo de estado reactivo con remember y mutableStateOf para implementar un contador funcional de reacciones ("Me gusta").

## 1.4 Objetivos específicos

- Crear e importar iconos vectoriales en el directorio res/drawable utilizando la herramienta Asset Studio.
- Configurar e importar las librerías necesarias de androidx.compose.runtime para la gestión del estado.
- Implementar una variable de estado reactiva con val counter by remember { mutableStateOf(0) }.
- Vincular el evento táctil mediante el modificador clickable para modificar el valor del contador.

- Estructurar la presentación gráfica dentro de contenedores LazyColumn y Row aplicando modificadores de dimensión y espacio.

### 1.5 Justificación

El dominio del estado en Jetpack Compose es imprescindible para la creación de cualquier aplicación Android comercial o académica de calidad profesional. Aprender a combinar recursos vectoriales optimizados con lógica reactiva permite desarrollar interfaces de usuario altamente responsivas, fluidas y visualmente atractivas.

## 4. Marco Teórico

---

### 2.1 Manejo de Estado en Jetpack Compose (remember y mutableStateOf)

En Compose, el estado determina qué se muestra en la interfaz en un momento dado. La función mutableStateOf() crea un objeto observable (State) que notifica a la interfaz cuando su valor cambia, desencadenando una recomposición. Por su parte, la función remember guarda y conserva este objeto en memoria para que no se reinicie a su valor predeterminado cada vez que el componente se vuelve a renderizar.

### 2.2 Vector Assets y Asset Studio en Android Studio

Asset Studio es una herramienta integrada en Android Studio que permite importar iconos vectoriales prediseñados (Material Icons) o archivos SVG/PSD y convertirlos al formato XML ejecutable por la plataforma Android (VectorDrawable). Los gráficos vectoriales garantizan una calidad visual nítida sin importar la densidad de píxeles o resolución del dispositivo móvil.

### 2.3 Componente Row e Iconos Interactivos

El composable Row organiza elementos de manera horizontal. Al colocar un elemento Image alineado junto a un componente Text dentro de una fila, se puede estructurar un módulo visual común en redes sociales: un icono descriptivo acompañado de su valor numérico o etiqueta.

### 2.4 Modificadores clickable y padding

El modificador clickable añade la capacidad de escuchar eventos táctiles sobre cualquier componente composable, ejecutando una función o expresión lambda cuando el usuario presiona el elemento. El modificador padding establece márgenes o separaciones internas/externas para mejorar la distribución estética.

### 2.5 Importación de librerías en Kotlin/Compose

Para trabajar de manera limpia con delegados de propiedades (operador by), se requiere la importación explícita de paquetes de tiempo de ejecución como getValue, setValue, mutableStateOf y remember pertenecientes al espacio de nombres androidx.compose.runtime.

## 5. Desarrollo

---

### 3.1 Importación de paquetes necesarios en MainActivity.kt

Al iniciar el desarrollo del proyecto en Android Studio, se revisaron y agregaron los paquetes correspondientes para soportar el diseño declarativo y la gestión de estado. Entre ellos destacan androidx.compose.runtime.remember, androidx.compose.runtime.mutableStateOf, getValue y elementos de diseño como TextAlign, Color y Modifier.

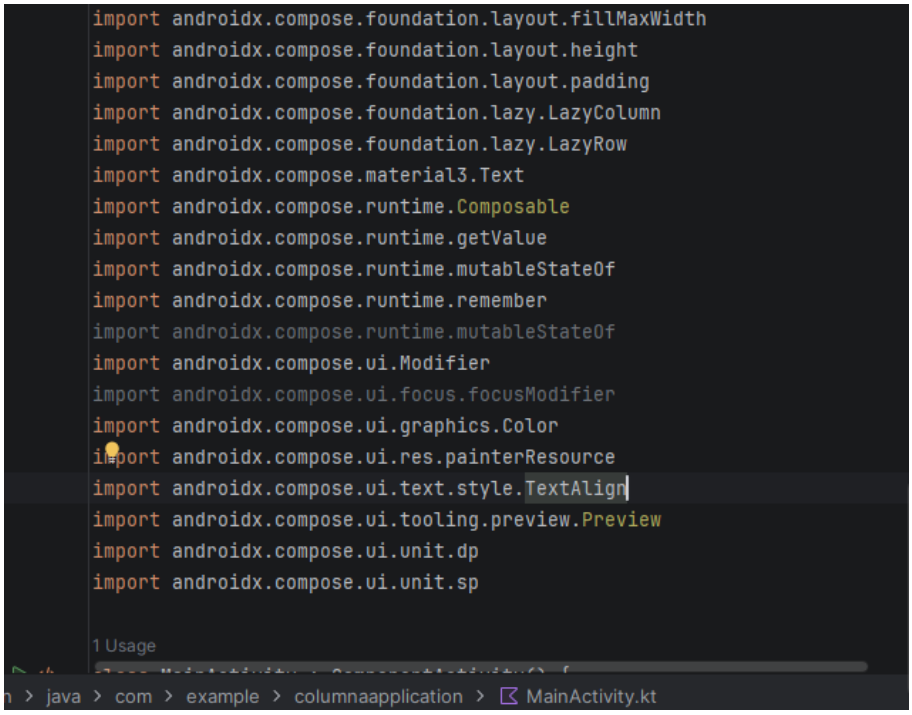



*Ilustración 1: Importación de librerías y paquetes en MainActivity.kt*

## 3.2 Apertura de Asset Studio para la creación de Vector Assets

Para incluir un icono gráfico personalizado en la aplicación, se hizo clic derecho sobre la carpeta res/drawable y se seleccionó New > Vector Asset. Esto abrió la interfaz de configuración de Asset Studio.

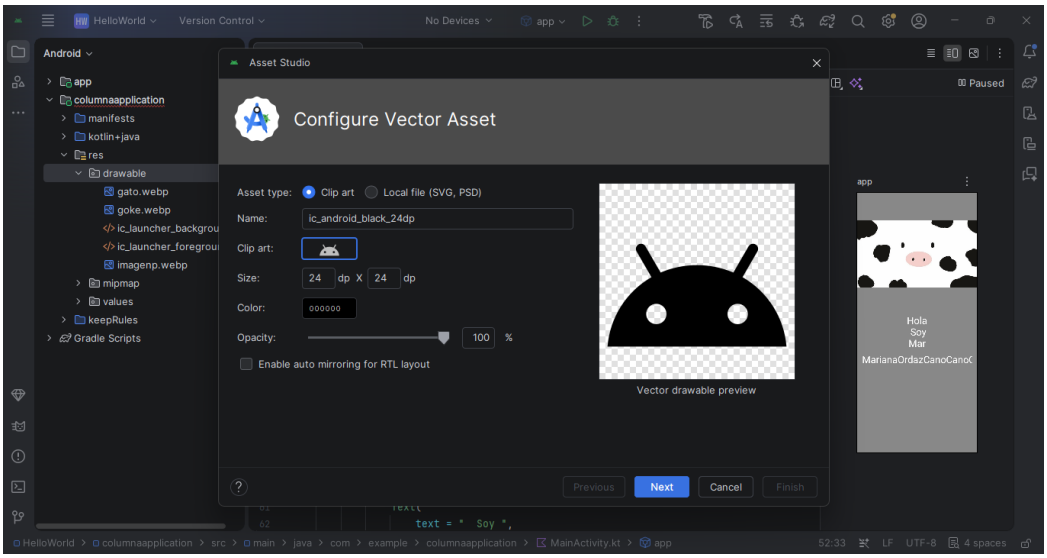



*Ilustración 2: Ventana de inicio de Asset Studio para la selección de Vector Assets*

### 3.3 Selección y configuración del icono de Me gusta (baseline_favorite_24)

Dentro del menú de Clip Art de Asset Studio, se buscó el icono de corazón correspondiente al nombre baseline_favorite_24. Se configuró con dimensiones de 24 dp x 24 dp y se completó el asistente de exportación para guardar el archivo ejecutable dentro de la carpeta drawable bajo la referencia R.drawable.favorite.

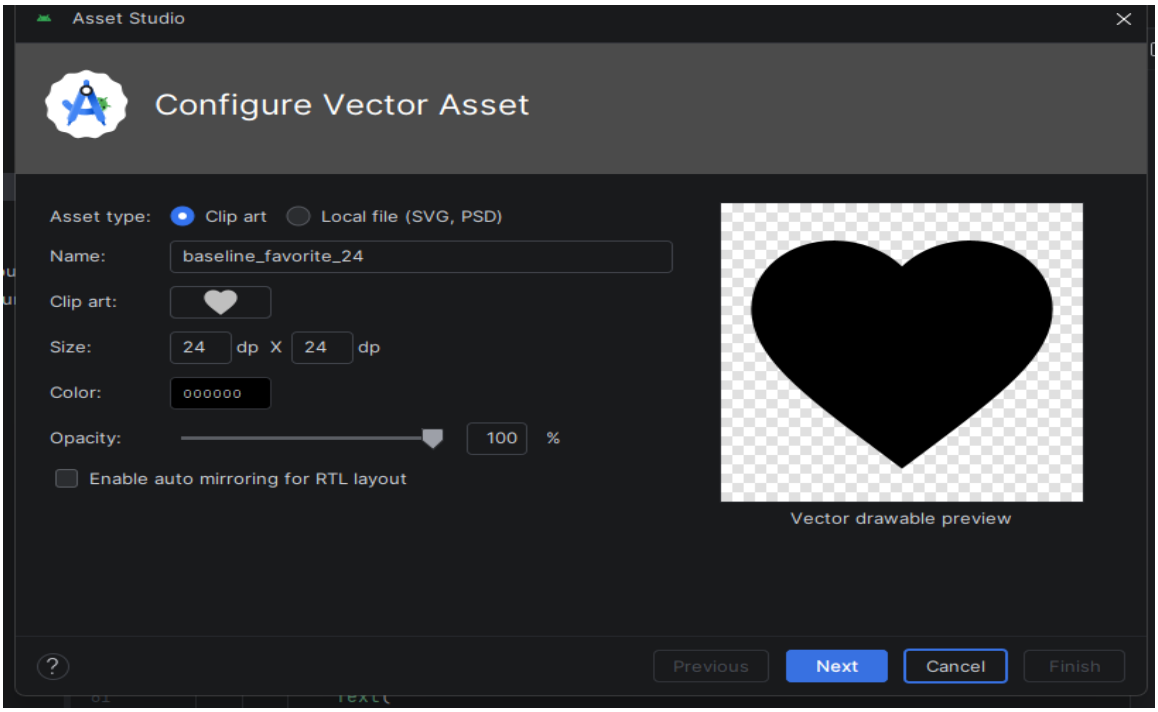



*Ilustración 3: Selección y personalización del icono de corazón (baseline_favorite_24)*

### 3.4 Declaración de la variable de estado para el contador (remember y mutableStateOf)

Dentro de la función composable principal app(), se definió la variable que rastrearía la cantidad de reacciones o clics del usuario mediante la línea de código: val counter by remember { mutableStateOf(value = 0) }. Esta variable conserva el conteo de clics a través de las recomposiciones de la interfaz.




*Ilustración 4: Declaración del estado del contador utilizando remember y mutableStateOf*

### 3.5 Integración del contenedor Row e icono interactivo

Se incorporó un componente Row al cual se le aplicó un margen superior mediante modifier = Modifier.padding(top = 8.dp). Dentro de esta fila se integró un componente Image que recupera el recurso gráfico cargado mediante painterResource(id = R.drawable.favorite) y una descripción accesible contentDescription = "like".

```
            contentDescription = "Imagen vaca"
        )
        Row{
            Image(painter = painterResource( id = R.drawable.favorite),
                contentDescription ="like"
            )
        }
        Text(
            text = "Hola",
            fontSize = 32.sp,
            color = Color.White,
            modifier = Modifier.fillMaxWidth(),
            textAlign = TextAlign.Center
```

*Ilustración 5: : Definición básica de la fila (Row) con el icono de Me gusta*

### 3.6 Vinculación del evento clickable para incrementar el contador

Para hacer el icono del corazón interactivo, se le añadió el modificador Modifier.clickable { counter++ }. Cada vez que el usuario presiona sobre el icono de corazón, la expresión lambda incrementa en una unidad el valor almacenado en la variable counter, provocando la actualización inmediata de los componentes vinculados.




*Ilustración 6: Adición del evento clickable en el icono para el incremento de contador*

### 3.7 Personalización visual del contador con Padding y Color.White

Al lado del icono en la misma fila, se ubicó un elemento Text cuyo contenido se vinculó al estado convirtiendo la variable numérica a texto mediante counter.toString(). Además, se

definieron los parámetros estilísticos color = Color.White y un margen de separación a la izquierda con modifier = Modifier.padding(start = 4.dp).

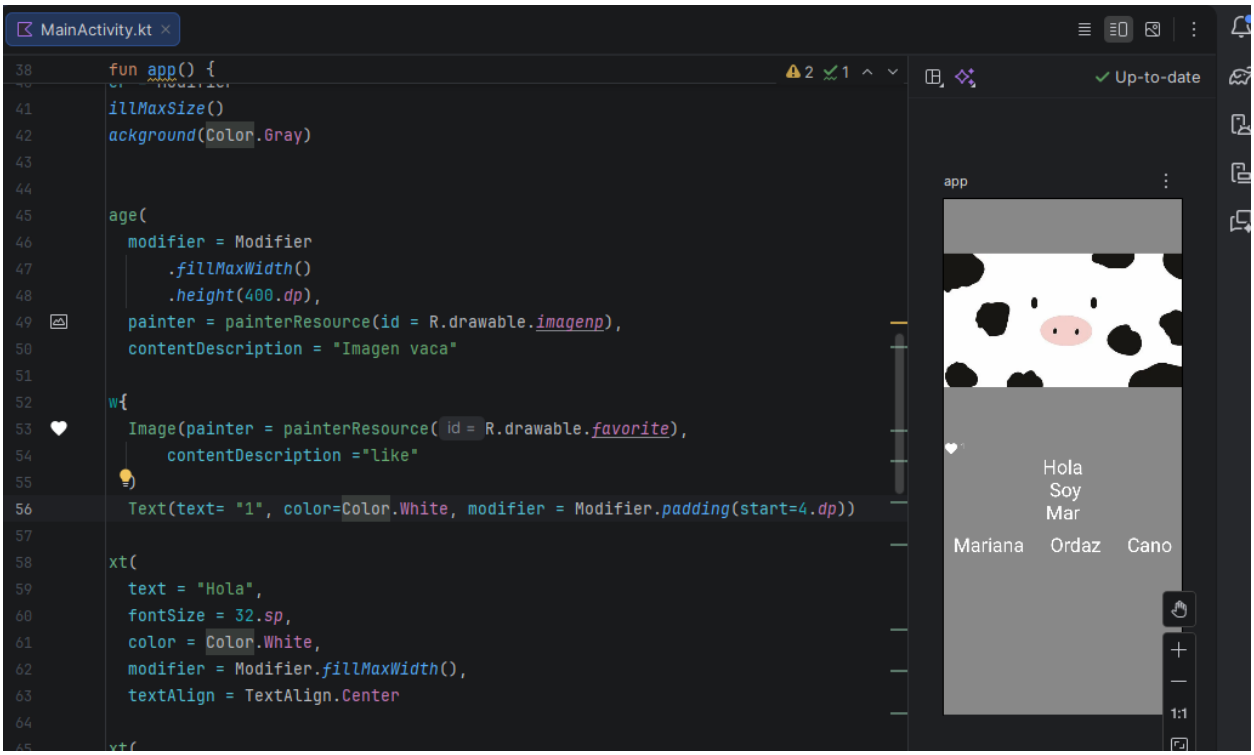



*Ilustración 7: : Inclusión del texto del contador con modificador de padding lateral*

## 3.8 Integración de la vista previa interactiva en Android Studio

A través del panel Interactive Preview de Android Studio, se probó la ejecución dinámica del componente. Al hacer clic sobre el corazón en la vista previa, el número mostrado cambió automáticamente de 0 a 1 (y sucesivamente), confirmando que la recomposición se ejecuta correctamente según lo esperado.




*Ilustración 8: Visualización previa interactiva del contador incrementado en Android Studio*

### 3.9 Estructuración completa del composable app()

Se unificó todo el bloque de código dentro del composable app() en el archivo MainActivity.kt, combinando el contenedor principal LazyColumn, la imagen de la vaca (imagenp), la fila interactiva con el contador de "Me gusta" y los bloques de texto de

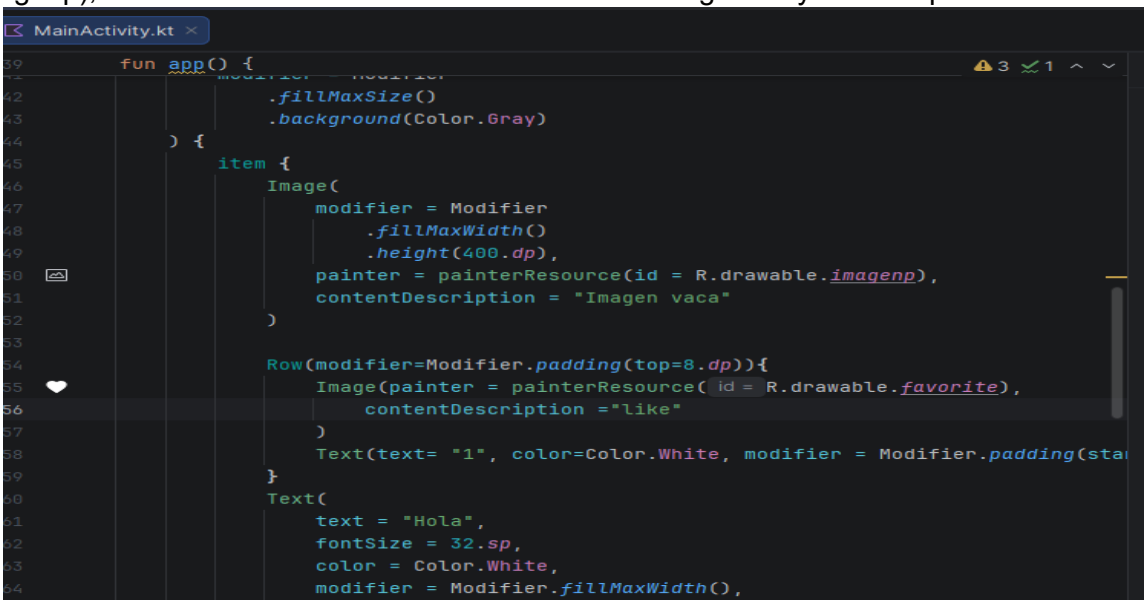



*Ilustración 9:Código estructurado completo de la función app()*

bienvenida alineados al centro.

### 3.10 Comparativa técnica de conceptos en Jetpack Compose

*Tabla 1: Comparativa entre conceptos de estado y recursos graficos*

| Concepto / Herramienta | Función Principal | Uso en la Práctica |
|---|---|---|
| **mutableStateOf()** | Crea un valor almacenable y observable que dispara la recomposición al cambiar. | Manejo dinámico del conteo de reacciones. |
| **remember { }** | Preserva el valor de un estado a través de recomposiciones sucesivas de la UI. | Evita que la variable counter se reinicie a cero al refrescar la pantalla. |
| **Asset Studio** | Convierte gráficos vectoriales e iconos de Material Design a archivos XML Android. | Generación del recurso vectorial baseline_favorite_24. |
| **Modifier.clickable { }** | Otorga capacidad de respuesta táctil y disparo de eventos a elementos visuales. | Incremento del valor de la variable counter++ al tocar el corazón. |

*Ilustración 10: comparativa entre conceptos de estado y recursos graficos*

## 6. Conclusiones

---

### 4.1 Aportes y limitaciones

**Aportes:** La práctica permitió entender de manera clara el flujo reactivo de Jetpack Compose. El uso de remember y mutableStateOf demostró cómo se pueden crear elementos interactivos como botones de reacción o contadores de forma sencilla y eficiente, sin necesidad de manipular manualmente referencias de la interfaz como se hacía en Vistas XML. Asimismo, el uso de Asset Studio facilitó la integración de iconos vectoriales optimizados.

**Limitaciones:** El estado configurado en esta práctica es local y reside únicamente en la memoria volátil de la aplicación. Esto significa que si la actividad se destruye o la aplicación se cierra, el contador vuelve a su estado inicial de cero al no contar con persistencia en base de datos local o remota (Room o Firebase).

### 4.2 Recomendaciones y proyecciones

Como siguiente paso, se recomienda explorar el uso de rememberSaveable para preservar el estado incluso ante cambios de configuración del dispositivo (como la rotación de pantalla). Adicionalmente, se sugiere integrar patrones de arquitectura más robustos como ViewModel y StateFlow para separar la lógica de negocio de la capa de interfaz de usuario.

## 7. Anexos

---

https://github.com/marianaordz12/-APLICACIONES-MOVILES

## 8. Referencias


- ---

AristiDevs. (2021, 12 de agosto). *Curso JETPACK COMPOSE desde CERO - CREA TU PRIMERA APP - Tutorial Android COMPOSE* [Video]. YouTube.

- Google Developers. (2024). *State and Jetpack Compose*. Android

Developers.

- Google Developers. (2024). *Add multi-density vector graphics in Android Studio (Asset Studio)*. Android Developers.